LAB ASSIGNMENT

NLP Foundations: Handwriting OCR Evaluation

Deepening your understanding after Lab 6

Course	Artificial Intelligence
Practical	Lab 6 of 8 - Handwriting OCR Evaluation
Student details	Name: **Tuan Nur Afifah Khaira Zulfa Binti Tuan Zaki** Student ID: **24006290**

Purpose

This assignment is to be completed after the Lab 6 practical. You do not need to repeat the entire experiment. Instead, use what you learned to explain the concepts, interpret the results, reason about OCR errors and suggest improvements. Answer in your own words.

Instructions

1. Complete the assignment individually.
2. Use your Lab 6 practical outputs and the results discussed during the lab as evidence.
3. Show all calculation steps. Round numerical answers to 2–3 decimal places where appropriate.
4. You may refer to your lab results; you do not need to rerun the complete OCR experiment.
5. Include screenshots or small tables only where requested. Ensure that all evidence is readable.

Part A - Understand the concepts

Q1. NLP pipeline

Describe the five stages of an NLP pipeline: acquire, normalise, represent, model and evaluate. For each stage, give an example of what happens when a handwritten medicine-name image is processed.

1. **Acquire**: Capture handwritten prescription image from dataset.
2. **Normalise**: Convert text string to lowercase and strip punctuation.
3. **Represent**: Convert cleaned text into character vectors or N-gram sequence tokens.
4. **Model**: Run Tesseract engine to predicct character sequence from image.
5. **Evaluate**: Calculate Edit Distance, CER, and Exact-Match Accuracy against ground truth.

Q2. How OCR works

Explain the difference between segmentation and character recognition. Give one example of a failure at each stage. Why is it useful to identify which stage caused an error?

Difference: Segmentation isolates individual character bounding boxes, while Character Recognition classifies the issolated pixels into alphanumeric characters.

Failure Examples:
- Segmentation: Slanted cursive letters merged together without gaps, causing missed characters.
- Character Recognition: Clear letter "m" misclassified as "d" due to noisy stroke pixels

Usefulness: Pinpoints whether the model failed to crop the text area or misread the character shape, guiding targeted preprocessing fixes.

Q3. Text normalisation

The reference is "Tab Azimax 40mg" and the OCR output is "tab azimax 40 mg". Apply the Lab 6 normalisation rule (lowercase and keep only letters and digits). Show both normalised strings. Why must the same rule be applied to both?

- **Reference**: "Tab Azimax 40mg" -> "tabazimax40mg"
- **OCR Output**: "tab azimax 40 mg" -> "tabazimax40mg"
- **Why same rule**: Ensures evaluation measures character recognition accuracy rather than formatting or spacing variations.

Q4. Normalisation trade-off

The Lab 6 normaliser removes spaces. Describe one situation where removing spaces could hide an important difference. Suggest how you would change the normalisation rule for that situation.

- **Situation**: Removing spaces hides boundaries between drug names and dosage (e.g., "Tab 50mg" vs "Tab50 mg" both become "tab50mg", masking spacing errors)
- **Fix**: Retain single whitespace delimiters while stripping non-alphanumeric special characters.

Part B — Work with OCR measurements

Q5. Levenshtein distance

Calculate the Levenshtein distance between "breeky" (reference) and "beeky" (prediction). Identify the edit operation(s). Then calculate the normalised edit distance by dividing the distance by the length of the longer string.

1. **Strings**: Reference = "breeky" (length 6), Prediction = "beeky" (length 5)
2. **Edit Operation**: 1 Deletion (remove 'r')
3. **Levenshtein Distance**: 1
4. **Normalised Edit Distance**: 1/6 = 0.167

Q6. Character Error Rate (CER)

An OCR system produces 4 substitutions, 2 deletions and 1 insertion. The reference contains 35 characters. Calculate CER using CER = (S + D + I) / N. Explain what the result means. Can CER be greater than 1? Explain briefly.

CER = (S + D + I) / N = (4 + 2 + 1) / 35 = 7/35 = 0.200 (20.0%)
- **Meaning**: 20% of character predictions required correction relative to reference length.
- **Can CER > 1?** Yes. If total insertions, deletions, and substitutions exceeds reference length N, CER exceeds 1.0.

Q7. Compare the metrics

Explain exact-match accuracy, mean normalised edit distance, CER and blank predictions. For each metric, state whether a higher or lower value is generally better. Why is exact-match accuracy alone insufficient?

Metric	Direction
Exact-Match Accuracy	Higher is better
Mean Normalised Edit Distance	Lower is better
Character Error Rate (CER)	Lower is better
Blank Predictions	Lower is better

- **Why Exact-Match alone is insufficient**: Binary pass/fail metric. It treats an output missing 1 character identical to a completely wrong output, hiding minor improvements.

Part C — Interpret the Lab 6 experiment

Use the results from Lab 6. Both pipelines were evaluated on the same 89 labelled images, with the same normalisation rule and fixed Tesseract configuration.

Q8. Read the results

Describe at least two improvements shown by the preprocessed results. Use values from the table as evidence.

- **Blank predictions**: Decreased from 39 to 15 images.
- **Character Error Rate (CER)**: Reduced from 0.875 to 0.788.

Q9. Explain the result pattern

CER and mean normalised edit distance improved, and blank predictions decreased. However, exact-match accuracy increased only from 0.000 to 0.011. Explain how preprocessing can help recover text without making the complete medicine name correct.

Preprocessing (grayscaling, borders, scaling) clarified character contours, allowing Tesseract to detect faint text regions and reduce blank outputs. However, since cursive letters remain interconnected and distorted, fine character recognition still fails, keeping exact matches low (0.011).

Q10. Controlled experiment

Why did the lab keep the Tesseract settings (--oem 1 --psm 7), the 89 images and the normalisation rule unchanged? What would be difficult to conclude if several settings changed at once?

- **Reason**: Isolates preprocessing as the sole independent variable.
- **Multiple changes consequence**: Cannot determine whether performance shifts resulted from image preprocessing or engine parameter tweaks.

Q11. Diagnose the failure

Identify the likely category for each case and explain:

(a) Tesseract raises an error while processing an image;
- **Engine Error**: Tesseract crashes due to unsupported file encoding or memory corruption during reading.

(b) OCR returns an empty string;
- **No Text Detected**: Poor contrast, severe blur, or faint handwriting causes low confidence threholds, outputting empty strings.

(c) OCR returns “nidu” when the reference is “nims”.
- **Character Confusion**: Visual similarity between handwritten cursive characters (e.g., “ims” resembles “idu”).

Part D — Apply your understanding

Q12. Suggest improvements

Suggest TWO improvements you would investigate next to improve handwritten OCR. For each, explain which error it targets and how you would test whether it helps. Do not claim an improvement works without evidence.

1. **Model Architecture Upgrade**: Replace standard Tesseract with a TrOCR (Transformer OCR) fine-tuned on handwritten datasets to resolve cursive segmentation failure.
2. **Adaptive Contrast Normalisation (CLAHE)**: Apply local histogram equalisation before OCR to improve faint ink visibility and eliminate background shadow noise.

Q13. Medical safety scenario

A clinic wants to use OCR to read medicine names and dosages from handwritten prescriptions. Explain at least THREE risks of using OCR output without human checking. Recommend a safer workflow.

- **Three Risks**:
1. Incorrect dosage recognition (e.g., 10mg reads as 70mg) causing overdose.
2. Drug name misinterpretation (e.g., Azimax reads as Azilect) administering wrong medication.
3. Blank prediction leading to missed treatment execution

- **Safer Workflow**: Implement a Human-in-the-Loop system where OCR auto-populates fields, requiring mandatory pharmacist visual verification before prescription processing.

Q14. Reflection

What is the most important thing you learned from Lab 6? Identify one concept you initially found difficult and explain it using an example from the practical.

- **Most Important Learning**: OCR performance depends heavily on structured pre-processing and target domain suitability (printed vs handwritten text).
- **Difficult Concept**: Normalised edit distance vs CER calculation. It was clarified by working through single-character operation matrices on prescription samples.

Submission to your AI Lab Git repository

IMPORTANT: Submit your completed assignment to the Git repository you created for the AI Lab.

Do not keep the assignment only on your computer or send it only through a private message. Upload it to your existing AI Lab Git repository.

6. Create a folder named Lab06_Assignment.
7. Save your completed report as Lab06_OCR_Assignment_YourName.pdf (or .docx).
8. Put supporting screenshots or figures in a subfolder named evidence.
9. Commit and push your changes to your existing AI Lab repository(if not done yet).
10. Open the repository online and confirm that your files are visible.

Suggested repository structure

```
AI-Lab-Repository/
└── Lab06_Assignment/
    ├── Lab06_OCR_Assignment_YourName.pdf
    └── evidence/
        ├── example_1.png
        └── results_table.png
```

Assessment focus

Your work will be assessed on conceptual understanding, correct calculations, interpretation of the actual Lab 6 results, quality of reasoning, clarity of writing and completeness of the Git repository submission.

Final reminder

The purpose is understanding, not simply reproducing code or copying results. Explain what the results mean, why the experiment was designed this way and what you would investigate next.